자율주행미들웨어응용

Autonomous Driving Middleware Applications

Jaerock Kwon, PhD
Associate Professor
Electrical and Computer Engineering
University of Michigan-Dearborn

Week 1

Middleware, drive-by-wire, and the ROS 2 computation graph

This course, in one slide

- You learn ROS 2 by **finishing a real vehicle**
- MRider: a 12 V ride-on car becoming a drive-by-wire autonomous platform
- The simulator already drives, maps, and navigates
- The vehicle **does not exist yet** — parts are being ordered now

Not exercises. Every deliverable closes a named gap in a real project.

What does not exist today

Missing	Status in the repository
`firmware/`	No firmware at all
`mitt_hardware`	The plugin that makes sim and real the same stack
Measured geometry	**"NOTHING IN THIS FILE HAS BEEN MEASURED"**
Real tests	29 tests exist. All 29 are linters

By December these exist because **you** built them.

The semester

Weeks		
0	**Last week**	ROS fundamentals — orientation
1–3	Foundations	Individual labs, everyone the same
4	**No class** — 개천절	Self-study + Lab 4
5	Description to motion	Lab 5
6–10	Four tracks	Electrical · Chassis · Simulation · Software
11–14	Merge ladder	Subsystems combine, then full integration
15	**Final demo**	12/21

1. What middleware is for

A robot is not one program

- Camera driver · LiDAR driver · localiser · planner · controller · safety monitor
- Different authors, different languages, different rates, sometimes different machines

ROS 2 gives you:

- Named, typed channels instead of ad-hoc sockets
- Discovery — processes find each other unaided
- Introspection — list, inspect, record, replay at runtime
- Python and C++ in one graph

What it costs you

Coupling that used to be a **compile error**
is now a **runtime condition.**

- Topic names must agree
- Message types must agree
- QoS profiles must agree
- Domain IDs must agree

When they disagree, mostly **nothing happens, and nothing complains.**

2. Drive-by-wire

Replacing the driver

Mechanical steering and throttle become **electrically commanded** equivalents.

- Steering → gearmotor + absolute angle sensor closing a position loop
- Throttle → motor driver taking a signed command

Three consequences follow, and they shape everything.

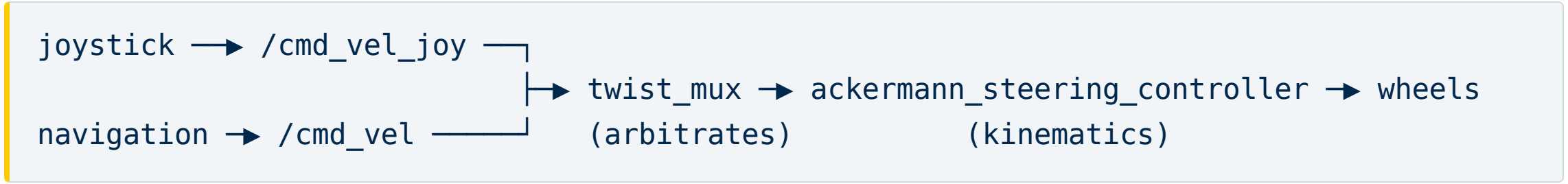
Three consequences

1. **Actuation is physical, and has limits**
   ±22.5° of steering. A rate limit. A stall current. Software cannot wish these away.

2. **Feedback is measured, not known**
   Every number the autonomy sees came from a sensor with noise, latency, and blind spots.

3. **A human must always be able to take it back**
   Not a feature — an architectural requirement outranking everything else.

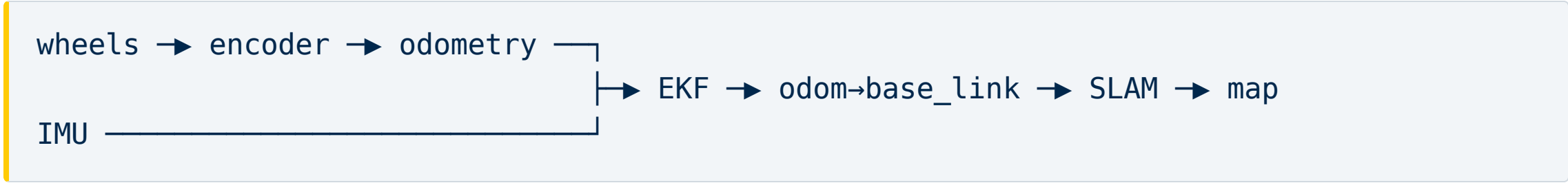
The command path



`twist_mux` priorities: **joystick 100**, navigation 10.

A human at the sticks wins.

The feedback path



- One encoder, on one motor shaft, of two motors wired in parallel
- Cannot see: gearbox backlash · tyre slip · the other wheel

Which is why it gets fused rather than trusted. (Week 6)

`twist_mux` is not the safety system.

And the design documents say so out loud.

Real authority is electrical

Layer	Works if the firmware has crashed?
Hardware E-stop — cuts traction	**Yes**
Relay MUX — reverts to stock steering	**Yes**
Hardware RC signal MUX	**Yes**
RC serial override into the controller	No

A software mux dies with the software it was meant to protect you from.

The principle

A safety mechanism that shares a failure mode
with the thing it protects against
is not a safety mechanism.

Software presenting itself as a safety system, then failing silently,
is worse than nothing — because people trust it.

3. Rates are contracts

Numbers you will keep meeting

Value	Meaning
`update_rate: 100`	Controller manager, so it is never the bottleneck
`reference_timeout: 0.5`	A command older than 0.5 s is not a command
≥ 50 Hz	Required DBW status rate from the vehicle
≤ 100 ms p95	Joystick-to-wheel latency budget

A system that meets its functional spec and misses its timing spec is **broken**.

On a vehicle, it is dangerous.

4. How ROS 2 fails

It fails by going quiet

Today you will open a terminal **without** sourcing `setup_env.sh`, and observe:

- **2 topics instead of 21**, and no nodes at all — the simulator is invisible
- **Exit code 0**
- **Zero bytes on stderr**
- Nothing in any log

The simulator is running perfectly the whole time.

The habit to build now

> *When something is **invisible** rather than **broken**,*
> *suspect the environment before you suspect your code.*

- A crash hands you a stack trace and a line number
- This hands you a **plausible wrong answer**: most of the graph is there, so it looks like one node crashed — and you go hunting for it

Weeks 3 and 4 add two more members to this family.

Lab 1 — Bring Up the Twin

1. Prove your environment — `check_env.sh` must report `FAIL: 0`
2. Launch the simulator; confirm both controllers `active`
3. Map the computation graph — `ros2 node list`, `topic info`, `rqt_graph`
4. Make it move — publish to `/cmd_vel_joy`
5. **Break it on purpose** — the unsourced terminal, and **partial** blindness

Deliverables: env check · graph dump · topic info · `rqt_graph.png` · written answers · AI declaration

Due before Week 2 (9/21)

Before you leave

- **Source `setup_env.sh` in every terminal** — it isolates your machine to loopback.
  It takes **two** variables: Gazebo ignores the ROS one entirely
- Read M1 — Intro to MRider & ROS 2
- If setup is still broken, **talk to me now**, not next week

Next week: nodes, topics, packages — and you write the first node this repository has ever had.